

COM-技术方案

摘要

本报告阐述我们在 FlagOS 开放计算全球挑战赛"长上下文 ICL 自动标注"赛道的技术方案。基于 Qwen3-4B + FlagScale (vLLM) 推理框架，我们为 8 个不同类型的标注任务（符号推理、语言计数、数学推理、字符串拼接、情感分类、自然语言推理、知识问答、Triton 代码生成）分别设计了专用的示例选择器、Prompt 模板和多轮交互策略。核心做法可以总结为四点：

- 挑相关的例子喂给模型**：用 BM25 算法在大示例池（1,899~5,500 条）中挑出和待标注样本最相关的若干条，再按任务结构特征（如列表长度、词性类型、Category 关键词、算子类型）分组排序，让最有用的例子放在离 input 最近的位置。赛题要求每条 prompt 的 ICL 上下文不低于 30K tokens（Task 8 为 16K tokens），我们精确控制在 30,000—31,000（Task 8 为 16,000—17,000）区间内，既满足下限合规又不浪费上下文窗口。
- 手把手把推理过程写给模型看**：所有需要 CoT 增强的任务（Task 1/2/3/4/6），统一采用 Qwen3 的 thinking 模式离线生成推理链（`think` 字段），且不暴露 GT 答案让模型独立推断，再用严格答案校验筛掉错的，确保留下的 think 都是正确推理（温度阶梯重试 0.0→0.3→0.6，重试耗尽置空）。在此之上，算法类任务（Task 1/3/4）额外叠加程序化 CoT（`cot` 字段），用代码精确计算确定性步骤（如"排序→逐对 diff→取 min"），形成 `think + cot` 双层示例增强。两层推理都作为示例的一部分嵌入 Prompt，让模型有"标准解题样板"可参考。
- 同一个题目让模型从不同角度答多遍再投票**：对分类和问答任务（Task 5/6/7），通过换 Prompt 模板、调整示例顺序、改变示例标签比例等方式，让模型在 `temperature=0` 的确定性条件下产出多个独立答案，再用多数票决定最终结果。
- 代码题用流水线一步步打磨**：对 Task 8（Triton 代码生成），让模型先出初稿，再通过静态校验找出错误、ReAct 多轮修复、风险评估、安全兜底改造等六个阶段层层把关，确保产出的代码可以顺利运行。

全部代码实现在 `COM/src/` 目录下，每个任务对应独立子目录。

一、问题定义与挑战分析

1.1 赛题目标

本赛题以 Qwen3-4B 大语言模型为核心驱动力，面向超长上下文条件下的自动化数据标注问题，要求对 8 个不同类型的数据集进行 In-context Learning 自动标注。赛题重点聚焦三个关键问题：

- 提示策略**：在超长上下文场景下，如何设计有效的模型指令与提示策略，引导 LLM 稳定、高质量地完成数据标注任务？
- 上下文构造**：当可用标注示例数量显著超过模型上下文容量时，如何为待标注数据构造信息密集、结构合理的超长上下文输入？
- 多轮交互**：在自动多轮对话或持续交互场景中，如何高效利用超长上下文，实现一致性与可扩展性兼顾的数据标注？

每个数据集提供大规模 ICL 示例池和测试样本，硬性约束为：Task 1–7 单条 prompt 的 ICL 上下文长度不低于 30K tokens，Task 8 不低于 16K tokens。

1.2 三大核心挑战

挑战一：提示策略——如何让模型在超长上下文下稳定产出高质量标注？

我们的做法：每个任务有独立的 `build_taskN_prompt()` 函数（位于各 `taskN/method.py`），根据任务特点定制 Prompt 结构和输出约束。关键策略包括：

- **Qwen3 离线 Think 生成 + 答案校验**：所有 CoT 增强任务（Task 1/2/3/4/6），统一让 Qwen3 thinking 模式独立产出推理链（不暴露 GT 答案），再用答案校验 + 温度阶梯重试筛掉错误的，只留正确推理作为 `think` 字段嵌入示例
- **额外程序化 CoT 叠加**：算法类任务（Task 1/3/4），在 `think` 之后追加代码精确计算的确定性步骤（如“排序→逐对 diff→取 min”）作为 `cot` 字段，形成双层增强
- **任务专属规则注入**：如 Task 2 的助动词黑名单（`AUX_VERBS` + `_VERB_RULES`）、Task 7 的 Jeopardy 格式约束（ClueTrap 检测 + Category 首字母校验）
- **多视角 Prompt 模板切换**：Task 5/6 的 `MULTI_VIEW_CONFIGS` 定义多个 `prompt_style`，每轮用不同角度的提示引导模型

挑战二：上下文构造——示例池远大于上下文窗口时，如何选出最有用的子集？

我们的做法：各任务 `method.py` 中的 `TaskNExampleSelector` 类统一实现，但根据任务输入特性采用不同的选择策略：

策略一：列表长度分桶（Task 1/3/4）——这三个任务的 input 是纯数字/字符列表，没有自然语言语义，BM25 无法有效区分。因此按输入列表长度分三级优先：同长度（priority=0）> 相邻 ±1（priority=1）> 其他（priority=2），组内按 input 长度短→长渐进排列，末尾放最相关的同长度示例。

策略二：BM25 语义检索（Task 2/5/6/7/8）——这些任务的 input 是自然语言句子或代码文本，语义相似度对标注质量有直接影响。纯 Python 实现 BM25（`k1=1.5, b=0.75`），按与待标注样本的文本相似度排序，低分→高分排列让最相似的示例紧贴 input（recency bias）。各任务在此基础上叠加额外维度：Task 2 按词性类型预过滤、Task 7 按 Category 三级语义匹配、Task 8 按操作类型分类优先。

共性机制：

- **Reverse-Select 反向截断**：从排序末尾反向累加 token 直到达到预算上限，确保高优先级/高相似度示例不被截掉
- **双边界精确控制**：`max_context_tokens=31000, min_context_tokens=30000`（Task 8 为 `17000/16000`），既达标又不浪费窗口

挑战三：多轮交互——如何利用多次调用提升标注一致性和准确率？

我们的做法：两种多轮范式覆盖不同任务类型：

- **多视角正交投票**：对分类和问答任务，通过换 Prompt 模板、调整示例顺序、改变标签比例等方式产出多个独立答案，`temperature=0` 保证确定性，多数票决定最终结果（Task 5: 5 轮 4 轴；Task 6: 3 轮 6 轴；Task 7: 3 策略 ensemble）

- 多阶段渐进式生成：对 Task 8（Triton 代码生成），六阶段 pipeline 层层把关：R1 初稿 → 静态校验 → ReAct 多轮修复 → R2 验证 → R3 安全兜底 → AST Guard，每个阶段修复上一阶段遗留的问题

二、系统架构与技术栈

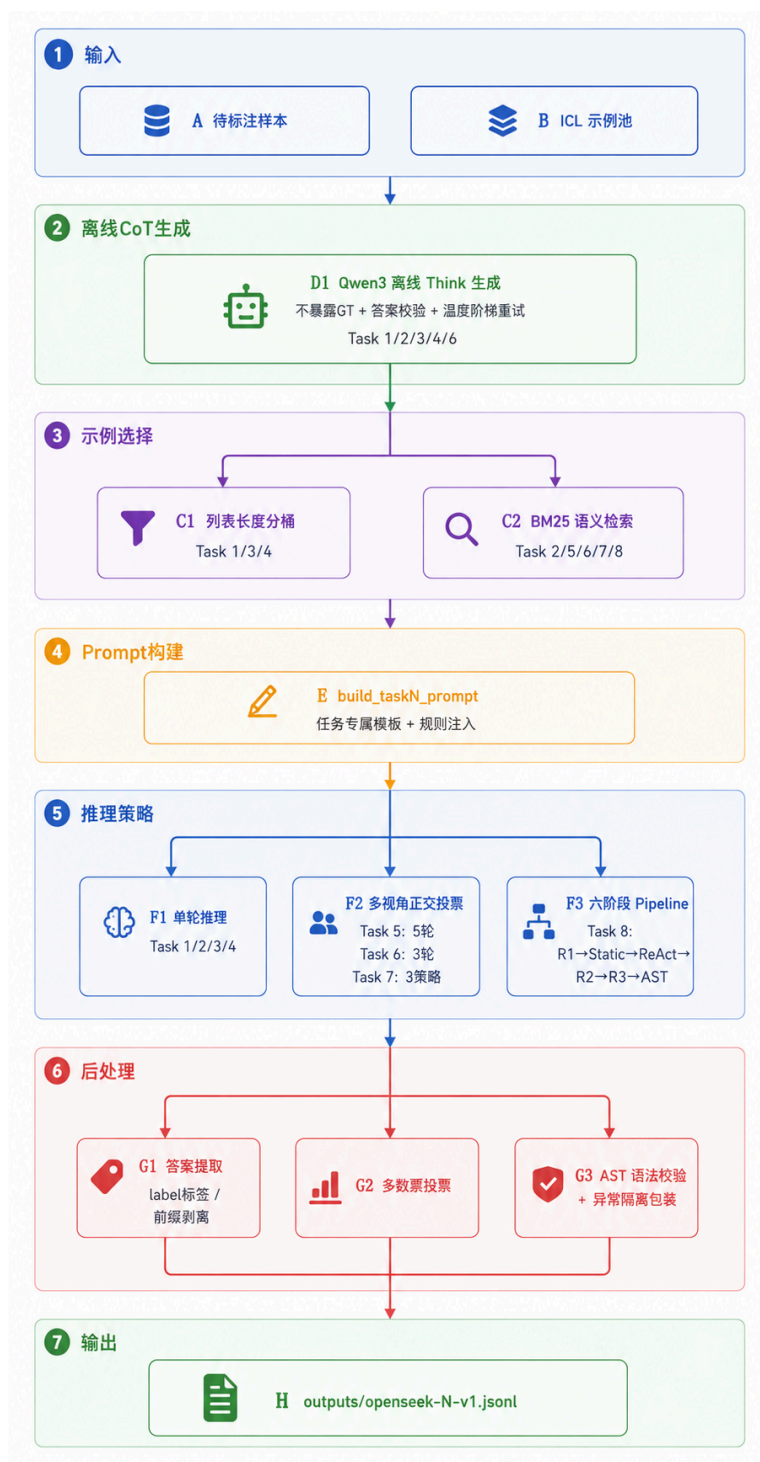
2.1 技术栈

组件	选型	配置来源
模型	Qwen3-4B	<code>env/llm_config.yaml: model: ../models/Qwen/Qwen3-4B</code>
长上下文	Qwen3-4B 原生 32K 上下文	模型自带 32,768 token 上下文窗口，未用 YaRN 做额外扩展
推理引擎	FlagScale (vLLM)	<code>env/llm_config.yaml: port=2026, gpu_memory_utilization=0.8, reasoning-parser=qwen3</code>
检索	Pure-Python BM25	各 <code>method.py</code> 中 <code>_BM25</code> 类 (<code>k1=1.5, b=0.75</code>)
Token 计数	Qwen3-4B Tokenizer	<code>transformers.AutoTokenizer.from_pretrained(MODEL_DIR</code>

2.2 开发环境

项目	配置
硬件环境	NVIDIA RTX 3090 (24GB) × 1, 内存 64GB
软件环境	Ubuntu 20.04, CUDA 12.8
Python	3.11.11 (conda 环境 <code>flagscale</code>)
推理服务	FlagScale vLLM, <code>gpu_memory_utilization=0.8</code> , 监听 <code>localhost:2026</code>

2.3 整体架构图



三、各任务详细技术方案

3.1 Task 1: closest_integers (最小相邻差)

任务定义：

给定整数列表，输出任意两数之间的最小绝对差值。

- 样本：500 条测试，5500 条 ICL 示例库
- 输入：JSON 字符串化整数列表，如 "[59, 26, -96, -30]"

- 输出：整数字符串，如 "33"

整体流程：

Markdown

ICL 示例库 (5500 条)
↓ 离线 CoT + think 生成 (generate_cot.py , 不暴露 GT + 答案校验)
带 cot + think 的示例库 (5484/5500 条 think 非空 , 99.7% 通过)
↓ 在线预测 (main.py)
├─ 列表长度匹配分桶 (Task1ExampleSelector)
├─ 排列 : priority 2→1→0 , 同长度紧贴末尾 (recency bias)
├─ 示例格式 : # input + think + cot + <label>
└─ 调用 : thinking 模式 + repetition_penalty=1.1 + temperature=0.0

离线 CoT + think 生成 (generate_cot.py)：

每条 ICL 示例产出两个字段：

- cot：程序化生成 (generate_cot_task1 函数)，排序→逐对 diff→取 Min→ <label>答案</label>
- think：Qwen3-4B thinking 模式自我推理，取自 reasoning_content

质量控制链：

措施	作用
Open-ended prompt	不暴露示例答案，杜绝锚点污染源
答案校验金标准	模型输出的 <label>X</label> 与标注答案严格一致才采纳
3 次重试 + 温度阶梯	0.0→0.3→0.6，低温失败时升温
scrub_think 二次清洗	删除残留答案锚点句
validate_think	非空 + 长度≥40 + 含 sort/diff/minimum 等关键字
校验未过即降级	think 留空，仅保留 cot，不污染 ICL

最终 5500 条示例中 5484 条 think 非空，校验通过率 99.7%。

示例选择 (Task1ExampleSelector 类)：

桶	优先级	拼接位置
同长度	0 (最高)	末尾 (紧贴 test, recency bias)
相邻长度 ±1	1	中段
其他长度	2	开头

- 组内排序：input 长度短→长 (渐进排列)
- Token 截断：reverse_select=True，从末尾反向累加到 31K，优先保留高优先级示例

程序化 CoT Scratchpad 示例：

HTML

```
Sorted (4): [-96, -30, 26, 59]
|-30 - (-96)| = 66
|26 - (-30)| = 56
|59 - 26| = 33
Min = 33
<label>33</label>
```

格式细节：`Sorted (N)` 加元素计数；每个 diff 展开实际数字；负数用括号包裹避免歧义。

推理参数：

参数	值	说明
<code>enable_thinking</code>	True	配合 think 示例使用 Qwen3 思考模式
<code>repetition_penalty</code>	1.1	抑制 thinking 模式下的死循环复读
<code>temperature</code>	0.0	确定性输出
<code>max_tokens</code>	2048	thinking 模式生成空间
<code>timeout</code>	600s	单条上限
<code>max_context_tokens</code>	31,000	约 28–40 条带 think 示例
<code>min_context_tokens</code>	30,000	赛题硬约束

Fallback 机制：

`content` 为空时从 `reasoning_content` 正则提取 `<label>X</label>` 兜底。

3.2 Task 2: count_nouns_verbs (词性计数)

任务定义：

- 给定英文句子，统计其中名词或动词的数量。
- 样本：500 条测试，约 5440 条 ICL 示例库 (noun/verb 各半)
 - 输入：`Sentence: '{sentence}'. Count the number of nouns/verbs in this sentence.`
 - 输出：整数字符串 (`<label>N</label>`)

整体流程：

Markdown

```
ICL 示例库 (~5440 条)
↓ 离线 CoT + think 生成 (不暴露 GT + 答案校验三连)
带 cot + think 的示例库 (约 85% 带 CoT, 其余降级保留 input+label)
↓ 在线预测
```

- └ 类型预过滤 (noun / verb 分池)
- └ filter_uncot=True : 过滤无 cot 降级样本
- └ BM25 句子相似度排序 (reverse_select : 最相似放末尾)
- └ 调用 : thinking 模式 + repetition_penalty=1.1 + temperature=0.0

离线 CoT + think 生成 (generate_cot.py):

- cot : 精简判定行 (如 Nouns: train, train, tracks ; Verbs: run, eat [skip: is, can])
- think : Qwen3-4B thinking 模式自我推理 (平均约 446 词)
- 答案校验三连: (1) <label>N</label> == GT; (2) 词列表数 == GT; (3) verb 列表不能落入助动词黑名单
- 约 85% 示例 think 非空; 其余 15% 降级, 预测阶段被 filter_uncot=True 过滤

示例选择 (Task2ExampleSelector 类):

步骤	说明
类型预过滤	根据 test 输入中 noun/verb 关键词, 取对应子池 (一次性缓存 + BM25 预建)
filter_uncot	过滤无 cot 降级样本 (约-15% 池规模); 池 < 100 时自动回落不过滤
BM25 评分	_extract_sentence() 提取纯句子做相似度排序, 相似度低→高
末尾排列	最相似放末尾 (recency bias), 紧贴 test
Token 截断	reverse_select=True , 31K 预算优先保留末尾高相关示例

单条 prompt 约容纳 46 条示例 (平均每条约 670 tokens 含 think)。

词性规则 (写入 Prompt 的硬约束):

规则	说明
助动词排除	is, are, was, were, am, be, been, being, do, does, did, will, would, shall, should, can, could, may, might, must
has/have/had	后接名词→计为动词; 后接过去分词→排除
-ing 形式	现在分词/动名词即使作形容词也计为动词
过去分词	被动/完成时作主动词时计入
名词	含专有名词, 单复数各算一次

推理参数:

参数	值	说明
<code>enable_thinking</code>	True	Qwen3 思考模式
<code>repetition_penalty</code>	1.1	抑制复读
<code>temperature</code>	0.0	确定性输出
<code>max_tokens</code>	4096	词性详细推理需更多生成空间
<code>timeout</code>	600s	单条上限
<code>max_context_tokens</code>	31,000	约 46 条带 think 示例
<code>min_context_tokens</code>	30,000	赛题硬约束

后处理：

多级兜底后由 `_validate_count` 确保输出为 $[0, 20]$ 区间整数，否则置空。

3.3 Task 3: collatz_conjecture (Collatz 猜想)

任务定义：

给定整数列表，对每个元素独立应用一次 Collatz 步骤（偶数 $n \rightarrow n//2$ ，奇数 $n \rightarrow 3n+1$ ），输出结果列表。

- 样本：500 条测试，3997 条 ICL 示例库
- 列表长度分布：2–9 个元素，均匀分布
- 输入：JSON 字符串化整数列表，如 `"[72, 29, 49]"`
- 输出：列表字符串，如 `"[36, 88, 148]"`

整体流程：

Markdown

```
ICL 示例库 ( 3997 条 )
  ↓ 离线 CoT + think 生成 ( generate_cot.py , 不暴露 GT + 答案校验 )
带 cot + think 的示例库 ( 3997/3997 条 think 非空 , 100% 通过 )
  ↓ 在线预测 ( main.py )
    ├── 列表长度匹配分桶 ( Task3ExampleSelector )
    ├── 排列 : priority 2→1→0 , 同长度紧贴末尾 ( recency bias )
    ├── 示例格式 : # input + think + cot + <label>
    └── 调用 : thinking 模式 + repetition_penalty=1.1 + temperature=0.0
```

离线 CoT + think 生成 (`generate_cot.py`)：

- `cot`：程序化逐元素 scratchpad (如 `72 even → 36; 29 odd → 88; 49 odd → 148`)
- `think`：3997/3997 条全部通过校验 (100% 通过率)

- 校验方式：从 `<label>[...]</label>` 提取整数列表，与 GT 逐元素严格比对

示例选择 (`Task3ExampleSelector`):

桶	优先级	拼接位置
同长度	0 (最高)	末尾 (紧贴 test, recency bias)
相邻长度 ± 1	1	中段
其他长度	2	开头

- 组内排序：input 长度短→长 (渐进排列)
- Token 截断： `reverse_select=True`，从末尾反向累加到 31K，优先保留高优先级示例
- 约 34 条示例/prompt (平均每条约 900 tokens 含 think)

Prompt 强约束 (`build_task3_prompt`):

- 强制逐元素展开 (`n even/odd → result`，每元素一行，保持原序)
- 严格 `<label>[result1, result2, ..., resultN]</label>` 收尾
- 禁止多次应用 Collatz / markdown / 自我纠正

推理参数:

参数	值	说明
<code>enable_thinking</code>	True	配合 think 示例使用 Qwen3 思考模式
<code>repetition_penalty</code>	1.1	抑制 thinking 模式下的死循环复读
<code>temperature</code>	0.0	确定性输出
<code>max_tokens</code>	4096	列表输出 + thinking 需更多生成空间
<code>timeout</code>	600s	单条上限
<code>max_context_tokens</code>	31,000	约 34 条带 think 示例
<code>min_context_tokens</code>	30,000	赛题硬约束

后处理:

`_validate_list` 确保输出能被 `ast.literal_eval` 解析为整数列表，否则置空。

3.4 Task 4: conala_concat_strings (字符串拼接)

任务定义:

给定字符串列表，按原顺序无空格拼接为单个字符串。

- 样本：500 条测试，3993 条 ICL 示例库
- 核心难点：大小写敏感 + 无空格拼接 + Qwen3-4B Tokenizer 倾向在 token 间插入空格

整体流程：

Markdown

ICL 示例库 (3993 条)

- ↓ 离线 CoT + think 生成 (3971/3993 think 非空 $\approx$ 99.4%)
- ↓ 在线预测
 - ├ 列表长度分桶 (同 Task 1/3)
 - ├ 示例格式 : # input + think + cot + `<label>带空格GT</label>`
 - ├ 调用 : thinking 模式
 - └ 后处理 : 智能去空格 (`postprocess_task4`)

关键设计——带空格拼接 + 后处理去空格 (tokenizer 友好策略)：

问题：大模型按 token 逐个预测的特性导致字符级拼接任务天然困难——模型倾向在 token 间插入空格、重复某些 token、混淆大小写字母，强制要求精确无空格拼接与模型生成机制冲突，错误率高。

解决：与其对抗 tokenizer，不如顺势而为——让模型输出带空格版本，由代码端后处理去除。

阶段	形态
ICL 示例 cot	<code>[1] "get" → get [2] + "B" → get B [3] + "have" → get B have</code> <code><label>get B have</label></code>
模型在线输出	<code><label>get B have</label></code> (带空格)
代码后处理	<code>postprocess_task4</code> : 去除元素间空格 → <code>getBhave</code>

后处理细节 (`postprocess_task4`)：

- 元素均不含空格 → 直接 `.replace(' ', '')`
- 有元素含空格 → 按列表元素顺序匹配，仅去元素之间空格，保留元素内部空格
- 元素无法定位 → fallback 全部去空格 (log warning)

Prompt 强约束 (`build_task4_prompt`)：

5 条 Critical Rules——保持原序 / 保持大小写 / 逐字符复制 / 保留标点 / 元素间加一个空格 (后处理自动移除)。与 CoT 示例格式一致的"带空格输出"约定，让模型不必同时承担"对抗 tokenizer + 保字符语义"的双重负担。

推理参数：

参数	值	说明
<code>enable_thinking</code>	True	配合 think 示例使用 Qwen3 思考模式
<code>repetition_penalty</code>	1.1	抑制 thinking 模式下的死循环复读
<code>temperature</code>	0.0	确定性输出
<code>max_tokens</code>	4096	拼接结果 + thinking 推理需更多空间
<code>timeout</code>	600s	单条上限
<code>max_context_tokens</code>	31,000	ICL token 上限
<code>min_context_tokens</code>	30,000	赛题硬约束

后处理：

兼容 16 种畸形标签组合 (`<label>` / `[label]` / `[label>` / `<label]` 及闭标签) 的正则提取 + `strip_label_wrapper` 剥离残留单边 prefix/suffix + `\boxed{...}` 兜底。

3.5 Task 5: tweet_sadness_detection (情感分类)

任务定义：

给定英文推文 (可能含 @mention、hashtag、emoji)，判断是否表达悲伤情感。

- 样本：500 条测试，1899 条 ICL 示例库 (Sad: 1121, Not sad: 778)
- 标签集： `{"Sad", "Not sad"}`

整体流程：

Markdown

```
BM25 索引 ( corpus/query 均剥离 @mentions 噪声 )
↓ 对每条 test 样本，跑 5 轮正交多视角投票
├─ R0 中性基线 Prompt
├─ R1 Not-sad default Prompt
├─ R2 Sad default Prompt
├─ R3 双面分析 In-Prompt CoT
├─ R4 心理学家三步 In-Prompt CoT
↓ 5 轮平权硬投票 ( majority_vote )
最终 prediction
```

多轮投票方案设计

- 四轴正交多视角设计：

轴	取值集合	作用维度
A 池构成	<code>full_top</code> / <code>top_mid_mix</code> / <code>top_bottom_mix</code> / <code>mid_bottom_mix</code>	信息相关性分布
B 标签比例	<code>natural</code> / <code>forced_50</code> / <code>reverse_40_60</code>	模型标签先验偏差
C 示例排序	<code>recency_up</code> / <code>recency_down</code> / <code>interleaved</code>	位置注意力偏差
D Prompt 框架	<code>neutral</code> / <code>not_sad_default</code> / <code>sad_default</code> / <code>cot_dual</code> / <code>psychologist_cot</code>	问法诱导偏差

每两轮在 A/B/C/D 上至少有 3 个轴不同，确保投票的正交独立性。

- 5 轮配置详表：

轮	A 池构成	B 标签	C 排序	D Prompt	max_tokens
R0	<code>full_top</code>	<code>natural</code>	<code>recency_up</code>	<code>neutral</code>	512
R1	<code>top_mid_mix</code>	<code>forced_50</code>	<code>interleaved</code>	<code>not_sad_default</code>	512
R2	<code>top_bottom_mix</code>	<code>reverse_40_60</code>	<code>recency_up</code>	<code>sad_default</code>	512
R3	<code>full_top</code>	<code>forced_50</code>	<code>recency_down</code>	<code>cot_dual</code>	1024
R4	<code>mid_bottom_mix</code>	<code>natural</code>	<code>interleaved</code>	<code>psychologist_cot</code>	1024

- 5 套 Prompt 模板（D 轴）：

模板	设计倾向	关键约束
<code>neutral</code>	中性基线	"Output ONLY the label wrapped in <code><label></code> tags"
<code>not_sad_default</code>	Not-sad 默认（保守）	"When in doubt, default to Not sad"
<code>sad_default</code>	Sad 敏感	关注 emoji/hashtag/word choice 微弱情感线索
<code>cot_dual</code>	双面分析 CoT	强制列举 Sad/Not sad 双向证据再裁决
<code>psychologist_cot</code>	心理学家三步 CoT	Step1 主情绪→Step2 是否属于 sadness family→Step3 输出

- In-Prompt CoT（R3/R4）：

`enable_thinking=False + temperature=0` 下靠 prompt 显式指定 Reasoning 输出格式，让 greedy 解码逐 token 生成推理链；不依赖外部 `cot_data` 文件。

- 投票策略：

5 票平权硬投票 (`Counter.most_common(1)`), 零先验、零拟合; 某轮返回 None 时跳过该票, 5 票全空才落空串。

示例选择器 (`Task5ExampleSelector`):

策略	配置	说明
检索算法	BM25 (<code>k1=1.5, b=0.75</code>)	基于词频的语义相关性排序
预处理	剥离 @mentions 噪声	corpus 与 query 均执行
Token 截断	<code>reverse_select=True</code>	从末尾反向累加, 稳定填充到 30.9K 附近

- 每轮独立检索, 保证各轮示例集不同 (正交性来自 Prompt/排列轴, 非示例集差异)
- 截断后约 50—60 条示例/prompt

推理参数 (5 轮共享):

参数	取值	说明
<code>temperature</code>	0.0	强制 greedy, 复现无偏
<code>enable_thinking</code>	False	关闭 thinking, 避免不可复现随机
<code>repetition_penalty</code>	1.0	不干预
<code>max_tokens</code>	512 / 1024	R0-R2=512; R3-R4 走 In-Prompt CoT 用 1024
<code>timeout</code>	600s	单次超时

3.6 Task 6: mnli_same_genre_classification (NLI 分类)

任务定义:

给定两个英文句子和一个 stated genre, 判断两句是否都属于该 genre。

- 样本: 500 条测试, 约 5500 条 ICL 示例库
- 标签集: `{"Y", "N"}`
- stated genre 取值: 10 类白名单 (`face-to-face / government / letters / 9/11 / slate / fiction / telephone / travel / oup / verbatim`)

整体流程:

Markdown

```
ICL 示例库 (~5500 条, 含离线 cot/think)
↓ 预处理 (构建 BM25 索引; 按 stated genre 建 genre 桶)
↓ 3 轮正交多视角投票
├─ R0 中性锚轮 (bm25_full / neutral / cot=off think=off / thinking=True)
```

```
├─ R1 保守压 Y (bm25_full / strict_n / cot=on think=on / thinking=True)
├─ R2 极端保守 (genre_filter / boost_n / interleaved / strict_n / cot=on think=on)
↓ Genre 白名单逻辑校正
↓ 3 轮平权硬投票 (平票取 N; 全 null 兜底 N)
最终 prediction
```

多轮投票方案设计

- 六轴正交多视角设计：

轴	取值集合	作用维度
A subset_mode	bm25_full / genre_filter / bm25_plus_genre	候选池来源 (BM25 vs genre 桶 vs 混合)
B label_balance	natural / 5050 / boost_y / boost_n	投喂示例的 Y/N 比例
C order	recency_up / recency_down / interleaved	位置注意力偏差
D prompt_style	neutral / strict_y / strict_n / inprompt_cot / linguist_cot	问法诱导偏差
E cot_inject	True / False	示例附带 3 行结构化 CoT
F think_inject	True / False	示例附带 reasoning think 段

- 3 轮配置详表：

轮	A subset	B balance	C order	D prompt	E cot	F think
R0	bm25_full	natural	recency_up	neutral	False	False
R1	bm25_full	natural	recency_up	strict_n	True	True
R2	genre_filter	boost_n	interleaved	strict_n	True	True

R0 提供"中性锚"，R1 叠 strict_n + 全注入做"保守压 Y"，R2 换 subset/balance/order 三轴做"极端保守"。

- 投票策略：

3 轮平权硬投票 + 平票取 N (保守策略) + 全 null 兜底 N (MNLI 多数类常识为 N)。

离线 CoT 数据 (generate_cot.py)：

- Qwen3-4B thinking=True 独立判断 sentence1_genre / sentence2_genre，不暴露 GT
- Y/N 统一推理 + GT 严格校验 + 3 次温度阶梯重试 (0.0→0.3→0.6)
- 产物： cot_data/openseek-6_mnli_with_cot.json (5500 条；约 82% 非空)

推理参数：

参数	值	说明
<code>enable_thinking</code>	True	开启 thinking 利用 reasoning_content
<code>repetition_penalty</code>	1.1	抑制 thinking 复读
<code>temperature</code>	0.0	确定性输出
<code>max_tokens</code>	2048	4 行结构化 + thinking 预算
<code>timeout</code>	600s	单条上限
<code>max_context_tokens</code>	31,000	ICL token 上限
<code>min_context_tokens</code>	30,000	赛题硬约束

后处理—Genre 白名单逻辑校正 (`_correct_by_genre_logic` 函数)：

模型输出 4 行结构化 (stated genre / sentence1 genre / sentence2 genre / `<label>`) 后：

- 若 s1_genre 与 s2_genre 均落在 10 类白名单内： `s1 == s2 == stated` → 强制 Y，其他 → 强制 N
- 否则放弃校正，回退到 `<label>` 解析
- 逻辑校正 在 5500 条数据上验证 99%+ 准确，能稳定纠偏模型 `<label>` 的 minority 偏差

3.7 Task 7: jeopardy_answer_generation (知识问答)

任务定义：

给定 Jeopardy 节目的 Category 和 Clue，生成对应的答案。

- 样本：500 条测试，5499 条 ICL 示例库
- 核心难点：知识面极广 + Category 是关键提示 + 间接提示需推理 + 7.6% 样本陷入思考复读

整体流程：

Markdown

```
BM25 + Category 三级分桶 ( 精确 / 模糊 / 其他 )
├─ 三策略并行检索
│   └─ A Category+Clue BM25 ( 三级分桶 + 四层 recency )
│   └─ B 纯 Clue BM25 ( 全量 5499 条 )
│   └─ C Category+随机打散 ( 同 A 分桶，桶内 seed=42 打散 )
├─ 三路 vLLM 调用 ( 统一 variant='A' Prompt，仅示例池不同 )
├─ 双道硬过滤 ( ClueTrap 整词陷阱 / Category 引号字母约束 )
├─ 全剔 → retry ( 带排除段重新生成 )
├─ 候选去重 → 多候选送验证器裁决
├─ 多数投票兜底
└─ 最终 prediction
```

示例选择

• 三策略 ICL 检索：

策略	候选池	排序	设计动机
A Category+Clue BM25	三级分桶（精确/模糊/其他）	BM25 + 四层 recency	主路：Category 强相关示例聚焦于末尾
B 纯 Clue BM25	全量 5499 条	BM25(query=Clue)	兜底路：Clue 语义召回
C Category+ 随机打散	同 A 分桶	桶内 random(seed=42)	正交路：消除 A 的 recency bias

• Category 三级语义匹配（`Task7ExampleSelector` 类）：

级别	匹配方式	排序
精确匹配	Category 字符串完全相同	BM25 相似度低→高
模糊匹配	去停用词后关键词重叠	重叠词数少→多
其他	剩余示例	BM25 相似度低→高

四层 recency 排列（A 策略）：其他→模糊→精确，末尾紧贴最相关示例（利用 LLM recency bias）。

特殊设计

• 双道硬过滤：

1. **ClueTrap 整词陷阱过滤**（`candidate_in_clue_trap()` 函数）：候选≥3 字符且在 Clue 中整词出现时剔除（Jeopardy 风格常见反例陷阱）
2. **Category 引号字母硬约束**（`parse_category_letter_constraint()` + `candidate_satisfies_letter()` 函数）：Category 形如 `"H" NAMES` 时，候选剥离冠词（a/an/the）后首字母须匹配

- **重调用机制**（`build_task7_retry_prompt`）：三路全被过滤时触发，附加排除段 `excluded=[...]` + 排除原因（`clue_trap | category_violation`），让模型避开已知错误候选。
- **验证器裁决**（`build_verifier_prompt`）：候选≥2 且分歧时调用，注入 strategy A 的 30K ICL 上下文作参考，模型输出 A/B/C 字母→自动展开为对应候选文本。单候选直接落选；候选词边界子串去重时短的胜出（如 `mafia` < `sicilian mafia` → 取 `mafia`）。
- **Thinking Fallback 链**：
 1. `content` 为空 → 从 `reasoning_content` 提取 `<label>` 标签
 2. 仍为空 → `rescue_from_thinking()`：匹配 `"the answer is X"` / `"final answer: X"` 等收敛模式

3. 仍为空 → 关闭 thinking 重发 (`enable_thinking=False, max_tokens=512, timeout=300`)

- 答案后处理 (`postprocess_task7`) : NFD 规范化去变音符 + lower + 末尾标点剥离。

推理参数:

参数	取值	说明
<code>temperature</code>	0.0	强制 greedy
<code>enable_thinking</code>	True	文字游戏推理需要 thinking
<code>repetition_penalty</code>	1.1	抑制复读循环
<code>max_tokens</code>	4096	thinking + 答案预算
<code>verifier_max_tokens</code>	2048	验证器调用更短
<code>nothink_max_tokens</code>	512	/no_think 降级时缩短
<code>nothink_timeout</code>	300s	降级时缩短超时

3.8 Task 8: kernel_generation (Triton 代码生成)

任务定义:

给定算子功能描述 + wrapper 签名 + Math 公式, 生成可执行的 Triton kernel + PyTorch wrapper 代码。

- 样本: 500 条测试
- 核心难点: 签名必须精确匹配 + 代码必须可执行 + Triton 编程模型复杂 + 单次生成可达上万 tokens
- 上下文预算: $16,000 \leq \text{ICL tokens} \leq 17,000$

整体流程 (`process_one_sample()`):

```
R1 Draft ( enable_thinking=True, temp=0.0, max_tokens=10000 )
↓
Static Validate ( 无 LLM: AST + 函数名 + 签名 + placeholder + exec 预检 )
↓ 失败 → 至多 3 轮
ReAct Repair ( 多轮 LLM, observation = 静态错误分类反馈 )
↓
R2 Verify JSON ( temp=0.3, 输出 risk_level + numerical_equivalence_confidence )
↓ safe + high/medium → 保留 draft
↓ 其他 → R3
R3 Safe-Exit ( enable_thinking=False, temp=0.0, max_tokens=6000 )
↓
AST Guard ( 无 LLM: wrapper 添加异常隔离层 )
```

Python

离线数据规整 (`normalize_dataset.py`):

由于提供的原始数据中，训练集与测试集的 input 格式差异比较大，且测试集 input 中存在单词拼写错误 (Trion)，所以我们对原始数据先做了预处理。

把原始数据一次性预处理为结构化字段: `func_name / func_signature / func_desc / math_formula / constraints / op_category`，运行时直接读取 `normalized_data/openseek-8_kernel_generation_normalized.json`。

示例选择 (`Task8ExampleSelector`):

- 检索源: normalized examples 的 `func_desc + math_formula`
- 同类型优先: 先按 `op_category` 同类示例 BM25 排序
- 跨类补充: 同类不够时按相似度跨类补
- Token 截断: `max_context_tokens=17000`，下限 16000
- `reverse_select`: 最相关示例放 prompt 末尾 (recency bias)

校验修复流程

- 静态校验 (`static_validate` , 4 道关):
 - AST parse: 代码必须能 `ast.parse`
 - Wrapper 函数名: 必须存在 `def {func_name}(...)`，参数名/顺序与 `func_signature` 一致
 - Placeholder 扫描: 检测 `TODO / FIXME / NotImplementedError / xxx / your code here` 等占位
 - Exec 预检: mock 环境 `exec` imports + kernel def，捕获语法/缩进错误
- ReAct Repair (`react_repair` , `max_rounds=3`):
 - observation prompt 包含: ICL 示例上下文 + 当前代码 + 静态错误清单 (前 8 条) + 函数签名硬约束
 - 仅修不重写，保持函数名/签名/大体结构不变
 - 退出条件: `static_validate` 通过 / 达到 `max_rounds`
- R2 路由决策 (`verify_routing_decision`):
 - R2 输出 `{risk_level, numerical_equivalence_confidence}` JSON (宽松 JSON 解析: 允许 markdown 包裹、单引号、尾随逗号)
 - `safe + high/medium + 0 hard_errors` → 保留 draft (`keep_draft`)
 - 其他 → 触发 R3 Safe-Exit (`safe_exit`)
 - 硬错误 (SyntaxError / 函数名缺失 / 签名不匹配) 直接 `safe_exit`
- R3 Safe-Exit (`build_safe_exit_prompt`):
 - 将 wrapper 函数体整体包进 `try / except Exception`，except 分支 return None

- 保留所有 import / `@triton.jit` kernel / wrapper 签名不动
- **AST Guard** (`apply_ast_guard`) :
 - Parse 成功且能找到 `def {func_name}` → wrapper 函数体外加异常隔离层 (`try: ... except Exception: pass`)
 - Parse 失败 → 生成最简 kernel 骨架 (imports + `@triton.jit` 空 kernel + 占位 wrapper), 保证代码结构合法
- **代码后处理**:
 - `fix_task8_imports()` : 自动补全缺失 import (math / typing / F / Tensor)
 - `postprocess_task8_wrapper()` : 确保 wrapper 函数名和参数签名与样本期望一致
 - `validate_prediction()` : 清理 markdown 围栏、检测重复输出、截代码起始位置

推理参数:

阶段	temp	enable_thinking	max_tokens	timeout	rep_penalty
R1 Draft	0.0	True	10000	900s	1.2
ReAct Repair	0.0	True	10000	900s	1.2
R2 Verify	0.3	False	4000	900s	1.1
R3 Safe-Exit	0.0	False	6000	900s	1.1

四、代码结构与复现

4.1 目录结构

```
COM/
├── src/
│   ├── common/                # 共享基础设施
│   │   ├── __init__.py
│   │   ├── llm_client.py      # LLM 调用封装 (annotate_nvidia / count_answer)
│   │   └── paths.py           # 路径中心 (VLLM_BASE_URL / MODEL_DIR / DATA_DIR)
│   ├── task1/                 # Task 1: closest_integers
│   │   ├── method.py          # 示例选择器 + Prompt 构建
│   │   ├── main.py            # 预测入口
│   │   ├── generate_cot.py     # 离线 CoT/Think 生成脚本
│   │   ├── cot_data/          # 带 cot+think 的数据
│   │   └── logs/              # 运行日志
│   ├── task2/                 # Task 2: count_nouns_verbs (同构 task1)
│   ├── task3/                 # Task 3: collatz_conjecture (同构 task1)
│   ├── task4/                 # Task 4: conala_concat_strings (同构 task1)
│   ├── task5/                 # Task 5: tweet_sadness_detection
│   └── method.py              # 多视角投票 + BM25 选择器
```

Python

```

├── main.py
├── logs/
├── task6/          # Task 6: mnli_same_genre_classification
│   ├── method.py  # 多视角投票 + Genre 逻辑校正
│   ├── main.py
│   ├── generate_cot.py
│   ├── cot_data/
│   └── logs/
├── task7/          # Task 7: jeopardy_answer_generation
│   ├── method.py  # 三策略 ensemble + 验证器
│   ├── main.py
│   └── logs/
├── task8/          # Task 8: kernel_generation
│   ├── method.py  # 六阶段 Pipeline
│   ├── main.py
│   ├── normalize_dataset.py # 数据规整脚本
│   ├── normalized_data/    # 规整后结构化数据
│   └── logs/               # 运行日志 + trace/
├── env/
│   ├── llm_config.yaml    # vLLM 启动配置
│   └── api_test.py        # API 连通性测试
├── scripts/
│   ├── run_all.sh         # 统一运行脚本 (支持选择性运行)
│   └── verify_outputs.sh  # 输出校验脚本
├── models/               # 模型目录 (放置 Qwen3-4B)
├── data/                 # 原始数据集
├── outputs/              # 预测输出 (openseek-{1..8}-v1.jsonl)
└── requirements.txt      # Python 依赖

```

4.2 环境准备

复现者可根据自己的环境情况灵活调整

```

git clone https://github.com/FlagAI-Open/OpenSeek.git
cd OpenSeek/openseek/competition/LongContext-ICL-Annotation
cd COM

### conda环境创建, 可选
conda init bash && source /root/.bashrc
conda create -n flagscale python=3.11.11 -y
conda activate flagscale

### 所需包安装
pip install -r requirements.txt -i https://pypi.mirrors.ustc.edu.cn/simple/

### Qwen3-4B模型下载
modelscope download --model Qwen/Qwen3-4B --cache_dir ./models

```

4.3 启动 FlagScale 推理服务

Bash

```
git clone https://github.com/FlagOpen/FlagScale.git
cd FlagScale
python run.py --config-path ../env --config-name llm_config action=run
python env/api_test.py # 验证模型是否已成功部署
```

vLLM 服务启动后监听 `localhost:2026`，提供 OpenAI 兼容 API。

4.4 运行全部任务

运行前，需修改 COM/src/common/paths.py 中的部分配置

- 1.COM_ROOT
- 2.MODEL_DIR（模型权重文件位置，若权重文件没按上述流程配置，则此处需修改）
- 3.VLLM_MODEL_ID（模型名称，与 llm_config.yaml 中保持一致）

```
bash scripts/run_all.sh          # 运行全部 8 个 task
bash scripts/run_all.sh 1 3 5    # 选择性运行
bash scripts/run_all.sh 3-7      # 范围运行
```

运行时间参考（单卡 4090D）：

Task	总耗时	均速	说明
Task 1	~2h 14min	16.1s/样本	单轮推理
Task 2	~1h 30min	10.8s/样本	单轮推理
Task 3	~2h 11min	15.7s/样本	单轮推理
Task 4	~1h 50min	13.2s/样本	单轮推理
Task 5	~3h 19min	23.8s/样本	5 轮投票
Task 6	~4h 31min	32.5s/样本	3 轮投票
Task 7	~6h 59min	50.3s/样本	三策略 + 验证器 + 重调用
Task 8	~4h 26min	96.0s/样本	六阶段 pipeline

4.5 校验输出

```
bash scripts/verify_outputs.sh
```

最终 8 份预测文件位于 `outputs/openseek-{1..8}-v1.jsonl`。

五、方案总结

- 1. Qwen3 离线 Think + 程序化 CoT 双层增强：所有 CoT 增强任务（Task 1/2/3/4/6）统一用 Qwen3 thinking 模式离线生成推理链（不暴露 GT + 答案校验 + 温度阶梯重试），算法类任务（Task

- 1/3/4) 额外叠加代码精确计算的确定性步骤 (`generate_cot_taskN()` 函数), 形成 `think + cot` 双层示例增强
2. **多视角正交投票**: 不依赖 temperature 采样, 通过 Prompt 模板 / 示例池构成 / 标签比例 / 排列顺序的正交变换产生独立视角 (Task 5: `TASK5_MULTI_VIEW_CONFIGS` 4 轴 5 轮; Task 6: `TASK6_MULTI_VIEW_CONFIGS` 6 轴 3 轮), 在 `temperature=0` 下实现多样性
 3. **结构特征驱动示例选择**: 列表长度分桶 (Task 1/3/4)、noun/verb 类型过滤 (Task 2)、Category 三级语义匹配 (Task 7)、算子类型分类 (Task 8), 超越通用 BM25 相似度
 4. **Reverse-Select Token 截断**: 利用 LLM recency bias, 从末尾反向填充 token, 确保最相关示例不被牺牲
 5. **六阶段代码生成 Pipeline** (Task 8): R1 Draft → Static Validate → ReAct Repair → R2 Verify → R3 Safe-Exit → AST Guard, 融合 LLM 生成与程序化校验
 6. **Thinking Fallback 链** (Task 7 `_call_variant()` 函数): content 提取 → rescue_from_thinking → /no_think 降级重发, 三级兜底确保答案回收
 7. **Tokenizer 友好策略** (Task 4): 顺应大模型 token 逐预测特性, 让模型输出带空格版本, 由 `postprocess_task4` 智能去除元素间空格, 避免对抗 tokenizer 造成的错误率激增
 8. **规则化逻辑校正** (Task 6 Genre 白名单 / Task 7 双道硬过滤): 利用任务领域确定性规则 (Genre 白名单判定 `_correct_by_genre_logic` / ClueTrap 整词陷阱过滤 / Category 引号字母约束) 对模型输出做规则化纠偏, 稳定提升准确率

* 本报告所有技术描述均可在 `COM/src/` 目录对应代码文件中直接验证。

* 为了验证方案的稳定性, 我们在本地 3090 和 autodl 4090D 机器上均已完成整套流程的复现, 所有环节日志均已保存